

DSA COACH

AI-Powered Data Structures and Algorithms Learning, Practice and Interview
Preparation Platform

PROJECT DOCUMENTATION AND IMPLEMENTATION PLAN

Team Name: AgentOps

Team Members

Isha Thapa
Jyotiraditya Mishra
Sneha Mohakud
Kuntal Shit

1. Project Overview

DSA Coach is an AI-powered Data Structures and Algorithms learning, practice, debugging, and interview preparation platform. The main objective is to provide users with a personalized digital DSA mentor that can help them learn concepts, practice coding problems, understand algorithms, debug programs, receive progressive hints, review code, track progress, and prepare for technical interviews. The system is designed as a production-oriented full-stack application rather than only a UI prototype. It follows separation of concerns, secure authentication, modular backend services, database management, isolated code execution, AI integration, testing, and scalable architecture.

2. Project Vision

The vision of DSA Coach is to act as an AI-powered personal DSA mentor that guides users from learning fundamental concepts to solving coding problems and becoming interview-ready. The platform will help users learn Data Structures and Algorithms, understand problem-solving patterns, practice DSA problems, receive progressive AI hints, understand brute-force and optimized approaches, write and execute code, identify programming and logical mistakes, receive AI-powered code reviews, track learning progress, follow an A2Z DSA roadmap, practice MCQs, prepare for interviews, interact with an AI DSA chatbot, and receive personalized recommendations.

3. Objectives

1. Create a centralized platform for DSA learning and practice.
2. Provide an interactive coding environment.
3. Provide secure execution of user-submitted code.
4. Integrate AI as a learning mentor rather than simply providing direct answers.
5. Provide progressive hints and problem-solving guidance.
6. Provide AI-based debugging and code review.
7. Track DSA learning progress.
8. Provide an A2Z DSA roadmap and MCQ practice.
9. Provide technical and behavioral interview preparation.
10. Maintain secure authentication and authorization.
11. Create a modular and scalable application architecture.

4. Scope of the Project

In Scope:

- User registration, login, Google authentication, Firebase authentication, password reset, profiles and settings.
- DSA topic-wise learning and A2Z roadmap.
- Approximately 200 initial DSA problems across major topics and difficulty levels.
- Search, filtering, bookmarks, problem status and submission history.
- Monaco code editor with Python, Java, C++ and JavaScript support.
- Public and hidden test cases, execution time and memory reporting.
- AI DSA chatbot, progressive hints, approach generation, code analysis, code review and code-fix suggestions.
- MCQ practice, interview preparation, mock interviews, tips and progress tracking.

Out of Scope for the Initial Implementation:

- Direct copying of existing platforms' UI.
- Independent password storage outside Firebase.
- Unrestricted code execution.
- Storing application data primarily in ChromaDB.
- Exposing AI API keys, database credentials or other secrets in frontend code.
- Presenting hardcoded responses as real AI functionality.

5. High-Level System Architecture

The proposed architecture is:

User → React Frontend → Firebase Authentication → Firebase ID Token → FastAPI Backend

FastAPI communicates with PostgreSQL, the AI layer, and a separate secure code sandbox. The AI layer uses LangChain, ChromaDB, RAG, embeddings and an LLM. PostgreSQL remains the source of truth for relational/application data, while ChromaDB is used for semantic retrieval.

6. Technology Stack

Frontend: React, Vite, TypeScript, React Router, Tailwind CSS, Axios, Firebase SDK, Monaco Editor, Lucide React.

Backend: Python, FastAPI, Uvicorn, Pydantic, SQLAlchemy, PostgreSQL, Alembic, Firebase Admin SDK, HTTPX, Redis and rate limiting.

AI: LangChain, LLM provider abstraction, embeddings, ChromaDB, RAG and structured AI outputs.

Testing: Pytest and Pytest Asyncio.

Code Quality: Ruff, type hints and environment-based configuration.

7. Project Structure

```
DSACoach/  
  frontend/  
  backend/  
  models/
```

The frontend contains the React application. The backend contains FastAPI APIs, services, authentication, database, AI and code-execution integration. The models directory contains AI/data/knowledge-base related assets and schemas.

8. Step-by-Step Implementation Plan

Phase 1 — Frontend Foundation: Initialize React + Vite, TypeScript, Tailwind CSS, routing, Axios, reusable components and layouts.

Phase 2 — Login and Signup: Build professional login and registration interfaces with email/password and Google authentication options.

Phase 3 — Firebase Authentication: Configure Firebase, AuthContext, login, signup, Google login, logout and password reset. Firebase remains the identity provider.

Phase 4 — Protected Routes: Protect dashboard, practice, problem detail, A2Z, MCQ, interview, tips and settings routes.

Phase 5 — Dashboard: Implement user greeting, statistics, DSA progress, AI Coach area, navigation sidebar and learning program cards.

Phase 6 — DSA Practice: Build the problem library with approximately 200 problems, topics, difficulty levels, tags and status.

Phase 7 — Search and Filtering: Implement search, difficulty, topic and status filters and sorting.

Phase 8 — Problem Detail: Build problem statement, examples, constraints, explanations, bookmark status and coding workspace.

Phase 9 — Monaco Editor: Add Python, Java, C++ and JavaScript support with editor tooling.

Phase 10 — Secure Code Execution: Create a separate execution service and isolated sandbox with CPU, memory, timeout, process, filesystem, network and output restrictions.

Phase 11 — Submission: Validate authentication, problem and language, execute hidden tests, store submission results and update progress.

Phase 12 — Progress Tracking: Track attempts, solved problems, accuracy, streaks, topic progress, difficulty progress and performance metrics.

Phase 13 — Database: Introduce PostgreSQL, SQLAlchemy and Alembic for users, problems, progress, submissions, bookmarks, conversations, MCQs, A2Z modules, interview questions and tips.

Phase 14 — A2Z Roadmap: Build structured DSA modules and calculate progress from actual solved problems.

Phase 15 — MCQ and Interview Systems: Add MCQ assessment and DSA, technical, behavioral, company-specific and mock interview preparation.

Phase 16 — AI Knowledge Base: Prepare DSA concepts, algorithms, patterns, complexity analysis, explanations, hints, mistakes and interview knowledge.

Phase 17 — ChromaDB and RAG: Clean and chunk knowledge, create embeddings, store semantic documents in ChromaDB, retrieve relevant context and pass it through LangChain to the LLM.

Phase 18 — AI Coach: Implement the DSA chatbot, progressive hints, problem approaches and personalized coaching.

Phase 19 — AI Code Assistance: Implement code analysis, code review and user-confirmed code-fix suggestions.

Phase 20 — Personalization: Use actual user activity to identify strengths, weaknesses, mistakes and learning needs.

Phase 21 — Testing: Test authentication, APIs, security, AI services and code execution.

Phase 22 — Security Hardening and Deployment: Apply rate limiting, validation, secret management, safe error handling, logging and production deployment.

9. AI Architecture

The AI flow will be:

User Message → Context Detection → Current Problem/User Code → Conversation Context → RAG Retrieval → Prompt Construction → LangChain → LLM → Structured Response → Frontend.

AI will be used for hints, explanations, approaches, debugging, code review, personalized coaching and interview simulation. Deterministic operations such as authentication, authorization, CRUD, progress, problem lookup and submission storage will remain normal backend services.

10. Security Requirements

The system will implement Firebase authentication, backend authorization, user data isolation, input validation, CORS, rate limiting, payload limits, secure code execution, error sanitization, secret management and request logging. Passwords, tokens, API keys, Firebase private keys, database credentials and other sensitive secrets must never be exposed in frontend code, committed to Git, or unnecessarily logged.

11. Testing Plan

Testing will cover authentication, protected routes, user isolation, problem retrieval, search, progress, bookmarks, chat validation, hint levels, code execution, submissions, rate limiting, AI services and security controls. Pytest and Pytest Asyncio will be used for automated testing. Testing will be performed continuously at every implementation phase rather than only at the end.

12. Development Milestones

Milestone 1 — Authentication Foundation: React → Firebase Login → Firebase ID Token → FastAPI → Firebase Admin Verification → User Synchronization → Dashboard.

Milestone 2 — DSA Practice: PostgreSQL → Problems → 200 DSA Questions → Problem APIs → Practice UI → Problem Detail.

Milestone 3 — Coding System: Monaco Editor → Run → Secure Sandbox → Test Results → Submit → Progress.

Milestone 4 — AI System: Knowledge Base → Embeddings → ChromaDB → Retriever → LangChain → AI Hint → AI Approach → AI Chat.

Milestone 5 — Advanced AI: AI Code Review → AI Code Analysis → AI Fix Suggestions → Interview Agent → Personalized DSA Coach.

13. Final Acceptance Criteria

Frontend: React application, professional UI, authentication, protected routes, dashboard, DSA practice, A2Z, MCQ, interview preparation, tips, settings, chatbot, problem interface, Monaco editor, code execution UI, AI assistance and responsive design.

Backend: FastAPI, Firebase token verification, PostgreSQL, SQLAlchemy, Alembic, user management, problem APIs, progress, bookmarks, submissions, secure code execution, MCQ/A2Z/interview/tips APIs and chat APIs.

AI: LangChain, LLM abstraction, ChromaDB, embeddings, RAG, knowledge ingestion, problem retrieval, hints, approaches, code analysis, code review and AI coaching.

Security: Authentication, authorization, user isolation, rate limiting, validation, sandboxing, secret protection and safe error handling.

Testing: Unit tests, API tests, authentication tests, security tests, AI service tests and code execution tests.

14. Conclusion

DSA Coach is designed as a complete AI-powered learning and interview preparation platform that combines DSA education, coding practice, secure code execution, AI assistance, progress tracking and interview preparation. The implementation will follow a phased development strategy beginning with the frontend and authentication foundation, followed by the backend, database, DSA problem system, secure code execution, AI knowledge base, RAG, LangChain integration and advanced AI features. The project prioritizes security, maintainability, modularity, scalability, testing and separation of concerns. The final goal is to provide users with an intelligent and personalized DSA mentor that helps them understand concepts, develop problem-solving skills, identify mistakes, improve coding ability and prepare effectively for technical interviews.